
Benjamin Adams

AN INVESTIGATION OF ULTRASONIC DETECTION ON A MICRO CONTROLLER IN A REAL TIME APPLICATION

Abstract – In this project an Arduino Uno R3 and three HC-SR04 ultrasonic sensors was used to show that by using just interrupts to create a scheduler, a working real time system can be implemented to receive data from the three ultrasonic sensors and then be able to detect the range of the object each sensor is scanning. Unfortunately, the project did not go as planned due to limited resources especially the lack of a sufficient power source that could properly power the ultrasonic sensors. The main idea behind the experiment was proven to work as a scheduler using just interrupts was able to detect signals from the ultrasonic sensors, however the sensors did not exhibit normal expected behavior and therefore detecting distance of objects with the ultrasonic sensors was not completed.

1 INTRODUCTION

For this project I want to use an Arduino connected to three sonar sensors to read the detection signals of the sonar signals with the Arduino using solely interrupts. I want to consistently and accurately detect signals from the sonar sensors and investigate if distance can be measured with the sonar sensors using this method. I want to see if it is possible to use interrupts to detect sonar readings from multiple sonars on an Arduino in a real time scenario.

2 BACKGROUND

In [1], [5], and [6] general info about Arduino is given. While [1] is a general introduction document to using Arduino in general, [5] and [6] are general resources and documentation about the Arduino UNO R3 specifically. However [1] is still useful for introduction as it complements [5] being an introduction and general source for the Arduino UNO R3 which then allows the contents of [6] to be navigated more easily when needing to find information for troubleshooting more in depth knowledge about the UNO R3.

The information in [2] as a source of general information for the HR-SR04 Ultrasonic sensor will be needed to understand how the sensor works with the Arduino using the information primarily from [5] and [6].

In [3] and [4] the hardware and circuit layouts are given for the UNO R3 which will probably be needed to some degree for getting the UNO R3 to physically interact with the ultrasonic sensors and other possibly needed hardware that may need to be connected.

The implementation examples of using the Arduino UNO R3 with the HR-SR04 ultrasonic sensor as shown in [7] and [8] together show how to implement both components of my project. While both [7] and [8] both have an Arduino working with an ultrasonic sensor, [7] has more emphasis on having the sonar and Arduino work together that will be needed for initial construction and implementation. [8] has more scope when it comes to working in a real time scenario with the Arduino where multitasking is needed which will be needed to take data from multiple sonars at the same time.

[9] will be important for most resources for code and general information when troubleshooting specifically as the git has code, examples, and specs on hardware and software. After being done with the introductory information in [1], [5], [6], and [2], [9] will be my most valuable source of information for finding information beyond what is general to complete the project.

[10] will be important as the Arduino IDE is the main reason why using an Arduino is a good choice for the project. Since much of the implementation of this project will be in the firmware to make the Arduino and ultrasonic sensors work with each other [10] in particular will be important for troubleshooting the Arduino IDE and getting it to work.

3 METHODOLOGY / METRICS

Since the goal of this project is to evaluate the feasibility of using three ultrasonic sensors with the Arduino. Most of the evaluation will be primarily be focused on the performance of the hardware and the results that can be gained with it. The project was done step by step to allow for me to learn as I go. For step 1, I planned to gain basic familiarity and figuring out general behavior of the Arduino Uno R3 and the HC-SR04 ultrasonic sensors. For step 2, I would build the circuit that would complete the physical implementation of the project as that would require basic knowledge. Step 4 was to experiment how to use the Arduino-IDE software and connect it to the Arduino. Step 5 was to create code to use interrupts to create a scheduler for the ultrasonic sensors and get signal detection implemented. Step 6 was detecting the distance of the object the sensor is detecting. Steps 1 through 4 were considered as pass or fail as they needed to be done in order to get to implementation. Step 5 has two metrics. The first was pass or fail for if sonar signals could be detected at all. The second metric was to test the consistency of accurate detection by testing the rates of false positives and false positives to true positives and true negatives. Step 6 would be measured by testing the accuracy of the sensor by calculating the relative error of the results.

4 RESULTS AND DISCUSSION

```
1 // ===== CONFIG =====
2 #define NUM_SENSORS 3
3
4 const uint8_t trigPins[NUM_SENSORS] = {2, 3, 4};
5 const uint8_t ledPins[NUM_SENSORS] = {10, 11, 12};
6
7 // ===== RANGE LIMIT =====
8 #define MAX_DISTANCE_CM 7 // 3 inches
9
10 // ===== STATE =====
11 volatile uint16_t startTime = 0;
12 volatile uint16_t endTime = 0;
13
14 volatile bool risingCaptured = false;
15
16 volatile uint8_t currentSensor = 0;
17
18 // Store raw pulse widths
19 volatile uint16_t pulseWidths[NUM_SENSORS];
20
21 // ===== LED STATE =====
22 enum LedState {
23     LED_OFF,
24     LED_ON,
25     LED_PULSE
26 };
27
28 volatile LedState ledState[NUM_SENSORS] = {
29     LED_OFF, LED_OFF, LED_OFF
30 };
31
```

Figure 1: Start of Code

In line 8 in Figure 1 is where the max distance of the sensor is set and clearly shows the value to be 7 cm which is around 3 inches.

```
93 }
94 }
95
96 // ===== SCHEDULER ISR =====
97 ISR(TIMER2_COMPA_vect) {
98     tickCount++;
99
100     static uint16_t lastTrigger = 0;
101
102     if ((tickCount - lastTrigger) >= SENSOR_PERIOD) {
103         if (!risingCaptured) {
104             ledState[currentSensor] = LED_ON;
105             triggerSensor(trigPins[currentSensor]);
106             lastTrigger = tickCount;
107             currentSensor = (currentSensor + 1) % NUM_SENSORS;
108         }
109     }
110 }
111 }
```

Figure 2: Scheduler code

Figure 2 shows the scheduler code which proves that the scheduler has been implemented and in a software level should work as intended.

In implementation of the experiment, I only got to step 5 of the initial plan. The first metric of step 5 technically passed and ultrasonic signals were being detected and scheduled with just interrupts as shown implemented in Figure 2. However, there was unexpected behavior from the circuit that stopped the process of the rest of the experiment. The only way to get the system to work was to limit the distance the sensors would send a signal from to about 3 inches as shown in Figure 1 and then the system would only work when an object was around a centimeter or less away from the sensor to the point the object is almost touching the sensor. This is not expected behavior and my only conclusion for the cause of this bug is a lack of a sufficient power source. Due to material limitations, I was using a USB Type-B cable to connect the Arduino to my pc to transmit data and provide power. Usually when using a HC-SR04 ultrasonic sensor with an Arduino, it is recommended to use an external 5V power source []. Considering that there are three ultrasonic sensors in the circuit there is little surprise that the sensors are not meeting power requirements for proper operation when the circuit is running. My only assumption for why detection and therefore the code works properly when reducing the range in the code is due to allowing just enough reduction of power consumption to allow the sensors to work at such a small range that doesn't even reflect the range defined in the code.

The rest of the code as well as the circuit diagram and picture of the circuit is in the

Appendix.

5 CONCLUSIONS

It was proven that the core of the experiment was technically successful, in that by using just interrupts, I was able to use an Arduino to read data from multiple ultrasonic sensors. However, some of the initial intent of the project was lost, and I wasn't able to use the ultrasonic sensors to detect varying ranges. For the most part the project works as intended and the core theory was proven to work. However, there were certainly issues with the operation of the ultrasonic sensors most likely due to an insufficient amount of power delivery to the ultrasonic sensors. There could also be issues with the code, but there is no way to know for certain without first fixing the power issue to rule that out. A major improvement to test out in the future for the experiment could be made in to retry the experiment with a sufficient power source and future research is needed to see if acquiring a proper power source is sufficient to allow full functionality of the hardware and allow to later on measure the accuracy of range measurements of the ultrasonic sensors.

6 REFERENCES

- [1] "Arduino Documentation – Fall 2015," Schoolpress, 2024. [Online]. Available: <https://schoolpress.sch.gr/stream/files/2024/01/Arduino.pdf>. (Accessed: 25-Apr-2026).
- [2] wmcclellan, "Inland HR-SR04 Blue Ultrasonic module 3PCS," Micro Center, [Online]<https://community.microcenter.com/kb/articles/660-inland-hr-sr04-blue-ultrasonic-module-3pcs> (accessed Apr. 25, 2026).
- [3] Micro I2C D19/SCL PC5 SCL D18/SDA PC4 SDA - arduino docs, <https://docs.arduino.cc/resources/pinouts/A000066-full-pinout.pdf> (accessed Apr. 26, 2026).
- [4] A000066-schematics.pdf, <https://docs.arduino.cc/resources/schematics/A000066-schematics.pdf> (accessed Apr. 26, 2026).
- [5] A000066-datasheet.pdf, <https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf> (accessed Apr. 26, 2026).
- [6] "Uno R3," docs.arduino.cc, <https://docs.arduino.cc/hardware/uno-rev3/?queryID=undefined#features> (accessed Apr. 25, 2026).
- [7] akk47, "Sonar using Arduino and display on processing IDE," Arduino Project Hub, <https://projecthub.arduino.cc/akk47/sonar-using-arduino-and-display-on-processing-ide-c0904b> (accessed Apr. 25, 2026).
- [8] Menphis, "Multitasking and real-time Arduino system," Arduino Project Hub, <https://projecthub.arduino.cc/Menphis/multitasking-and-real-time-arduino-system-9da221> (accessed Apr. 25, 2026).
- [9] "Arduino," GitHub, <https://github.com/arduino> (accessed Apr. 25, 2026).
- [10] "Arduino/Arduino-IDE: Arduino IDE 2.X," GitHub, <https://github.com/arduino/arduino-ide> (accessed Apr. 25, 2026).

7 APPENDIX

```
31
32 // ===== TIMING =====
33 volatile uint16_t tickCount = 0;
34 const uint16_t SENSOR_PERIOD = 30; // faster now (short range)
35
36 // ===== TIMER1 =====
37 void setupTimer1() {
38     TCCR1A = 0;
39     TCCR1B = 0;
40
41     TCCR1B |= (1 << CS11);
42     TCCR1B |= (1 << ICES1);
43
44     TIMSK1 |= (1 << ICIE1);
45 }
46
47 // ===== TIMER2 =====
48 void setupTimer2() {
49     TCCR2A = 0;
50     TCCR2B = 0;
51
52     TCCR2A |= (1 << WGM21);
53     TCCR2B |= (1 << CS22);
54
55     OCR2A = 249;
56
57     TIMSK2 |= (1 << OCIE2A);
58 }
59
```

Figure 3: Timing sections of the code

```
59
60 // ===== TRIGGER =====
61 inline void triggerSensor(uint8_t pin) {
62     digitalWrite(pin, HIGH);
63     delayMicroseconds(10);
64     digitalWrite(pin, LOW);
65 }
66
67 // ===== INPUT CAPTURE ISR =====
68 ISR(TIMER1_CAPT_vect) {
69
70     if (!risingCaptured) {
71         startTime = ICR1;
72         risingCaptured = true;
73         TCCR1B &= ~(1 << ICES1);
74     } else {
75         endTime = ICR1;
76
77         uint16_t width = endTime - startTime;
78         pulseWidths[currentSensor] = width;
79
80         risingCaptured = false;
81
82         // Convert to distance (cm)
83         float distance = width * 0.0343 / 2.0;
84
85         // ===== RANGE FILTER =====
86         if (distance <= MAX_DISTANCE_CM && distance > 0) {
87             ledState[currentSensor] = LED_PULSE; // valid detection
88         } else {
89             ledState[currentSensor] = LED_OFF; // ignore far objects
90         }
91
92         TCCR1B |= (1 << ICES1);
```

Figure 4: Sensor triggering and input caption portion of the code

```
117
118 // ===== SETUP =====
119 void setup() {
120
121     for (int i = 0; i < NUM_SENSORS; i++) {
122         pinMode(trigPins[i], OUTPUT);
123         digitalWrite(trigPins[i], LOW);
124
125         pinMode(ledPins[i], OUTPUT);
126         digitalWrite(ledPins[i], LOW);
127     }
128
129     pinMode(8, INPUT);
130
131     setupTimer1();
132     setupTimer2();
133
134     sei();
135 }
```

Figure 5: Initial setup portion of the code

```

136
137 // ===== LOOP =====
138 void loop() {
139
140   for (int i = 0; i < NUM_SENSORS; i++) {
141
142     switch (ledState[i]) {
143
144       case LED_OFF:
145         digitalWrite(ledPins[i], LOW);
146         break;
147
148       case LED_ON:
149         digitalWrite(ledPins[i], HIGH);
150         break;
151
152       case LED_PULSE:
153         digitalWrite(ledPins[i], LOW);
154         delay(20);
155         ledState[i] = LED_OFF;
156         break;
157     }
158   }
159 }

```

Figure 6: LED behavior management portion of the code

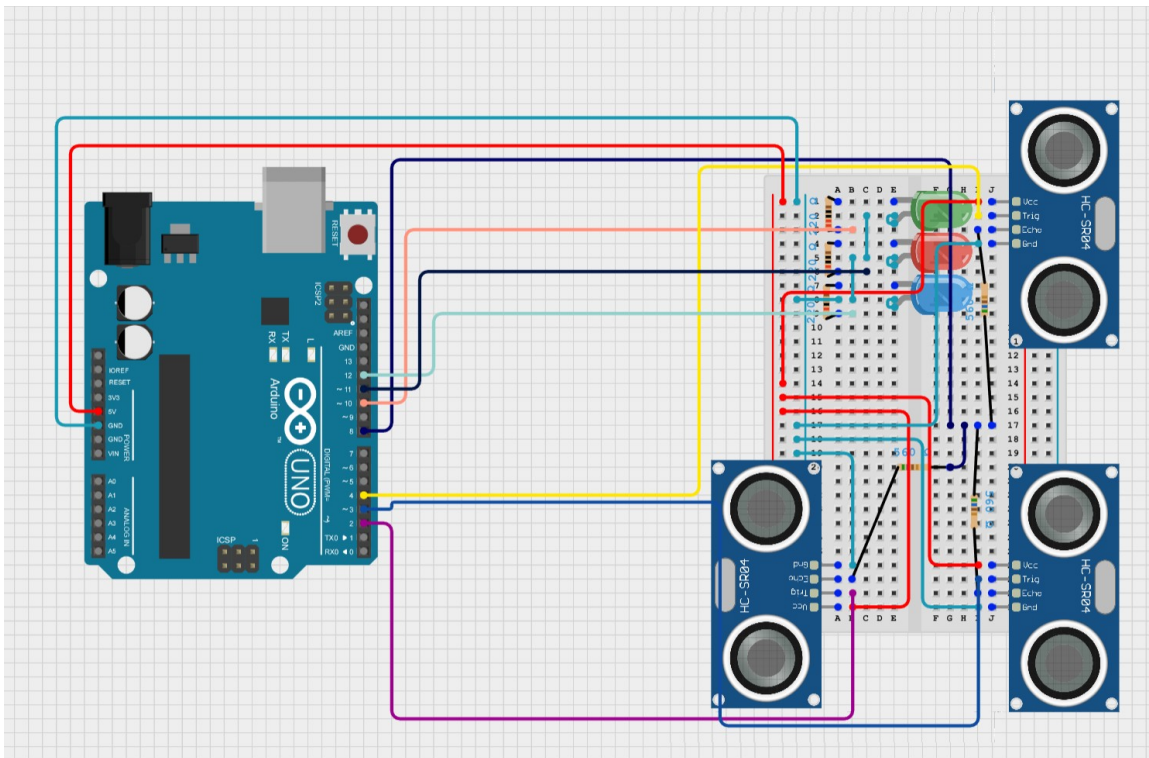


Figure 7: Diagram of Arduino and breadboard

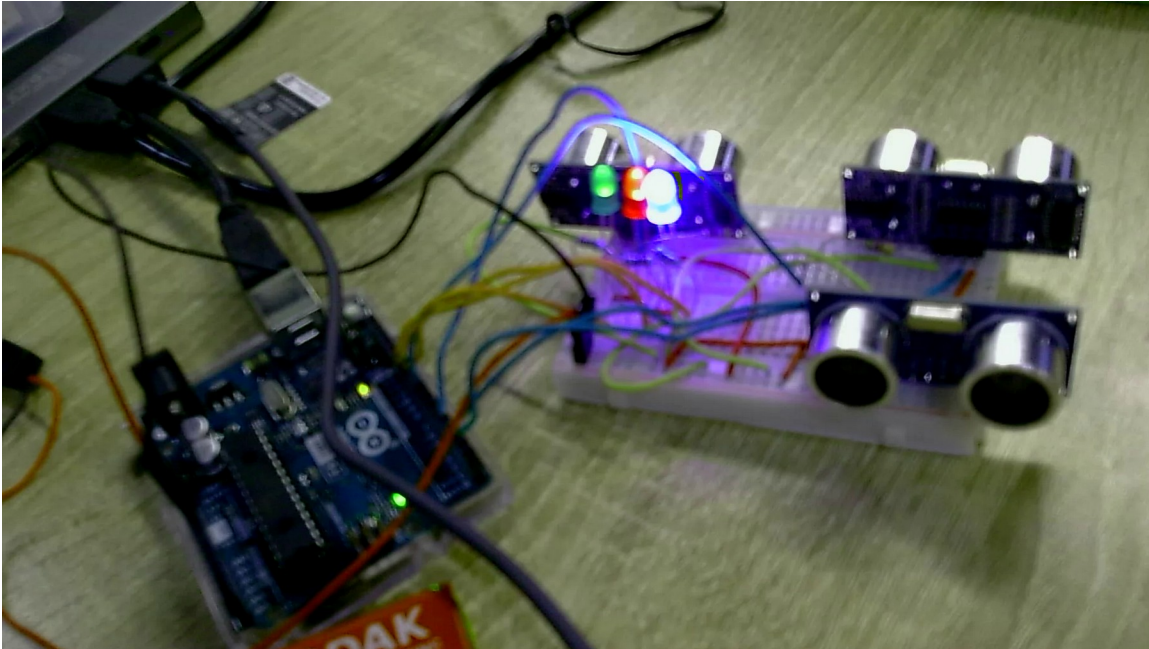


Figure 8: Picture of actual implemented circuit